

Front End Development

17-316/616 Fall 2025

AI Tools for Software Development

<https://ai-developer-tools.github.io>

Austin Henley and Andrew Begel

Today

- Project Team Meeting
- Course Feedback Results
- Turning a wireframe/mockup into front end code.
 - Mobbing

Project Team Meeting

- Spend the next 15 minutes to meet with your team.
- Work on P2: Development Specifications

Course Feedback

- Instructors Start
 - Sharing more wisdom from your industry experience. Any practical tips? What's going on in the AI startup world?
 - Increasing in-class team discussion time.
 - Analyzing case studies of software engineering where AI could help (or harm).
 - Posting additional resources to look at.
 - Posting grades more quickly.
 - Teaching us some basic software engineering information and tools.
 - Giving us more guidance on planning our projects.
 - Giving more time to discussants to talk about their thoughts before asking the class to offer theirs.
 - Feeding us more.
 - Adding more hands-on in-class activities.
- Instructors Stop:
 - Specifying the length requirement for reflection essays.
 - Requiring students to do all their work in groups.
- Instructors Keep:
 - Encouraging students to talk in class.
 - Asking to students to present reflections.
 - Mobbing
 - Connecting lectures, discussions, and homework

UI Development Big Picture

- UI Architecture
 - Monolithic, Modular, Micro-frontend
 - Model-View-Controller (MVC), Model-View-Presenter (MVP), **Model-View-ViewModel (MVVM)**
 - Using ChatGPT: What are the differences between MVC, MVP, MVVM?
- Technology Stack
 - HTML, CSS, client-side JavaScript or TypeScript
 - UI packages (React, Svelte)
 - Browser (Chrome, Safari, Firefox)
- Standards Compliance
 - Accessibility (WCAG/ARIA)

Component-Based Development

Front End User Interface

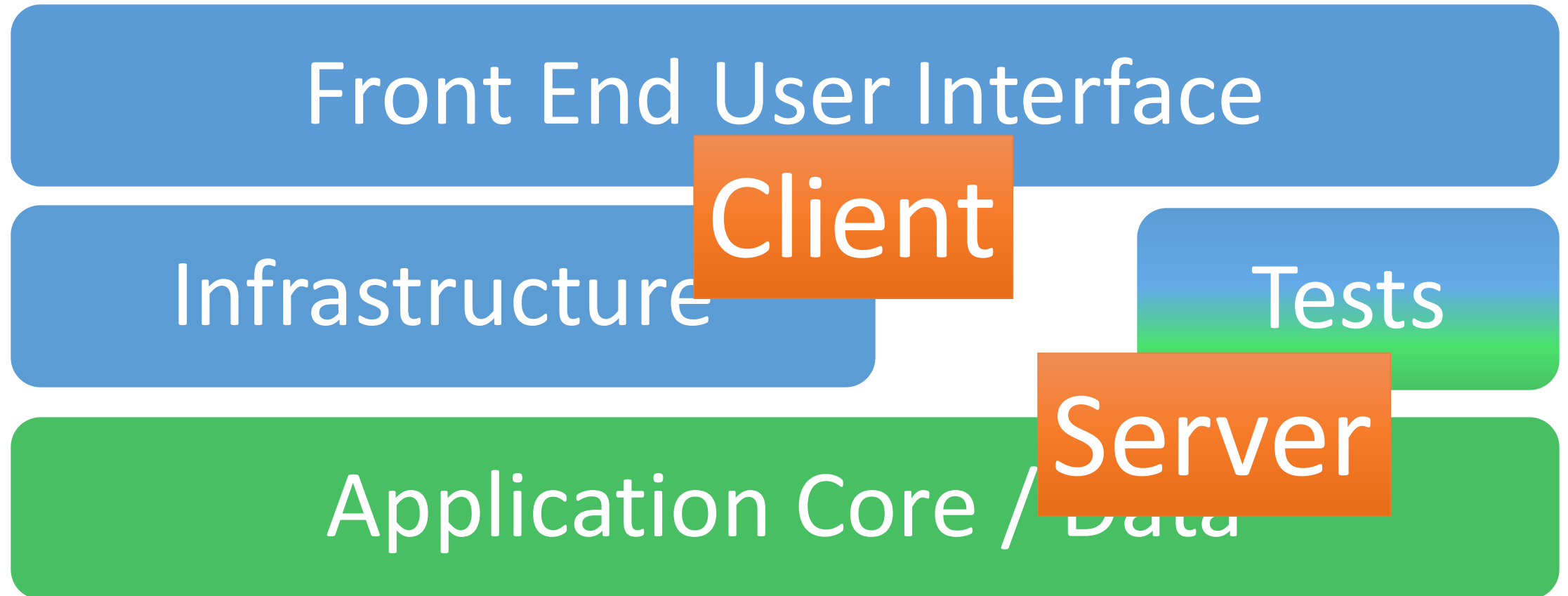
Infrastructure

Tests

Application Core / Data

Where do these components live?

Component-Based Development



Your app lists Instagram followers

```
void buttonClicked() {  
    render(getFollowers());  
}  
List<Friend> getFollowers() {  
    Connection c = http.getConnection();  
    InstagramAPI api = new InstagramAPI(c);  
    return api.getFollowers("john");  
}
```


Getting followers requires the Instagram API

```
void buttonClicked() {  
    render(getFollowers());  
}  
List<Friend> getFollowers() {  
    Connection c = http.getConnection();  
    InstagramAPI api = new InstagramAPI(c);  
    return api.getFollowers("john");  
}
```

But we haven't built our IG backend, yet

- What if Instagram is down?
- Or slow?
- Or you don't have John's credentials handy?
- Or you don't feel like vibe coding a clone of Instagram just so you can build your frontend?

```
void buttonClicked() {  
    render(getFollowers());  
}  
List<Friend> getFollowers() {  
    Connection c = http.getConnection();  
    InstagramAPI api = new InstagramAPI(c);  
    return api.getFollowers("john");  
}
```

Eliminate the backend dependency by mocking it

```
void buttonClicked() {  
    render(getFollowers());  
}  
List<Friend> getFollowers() {  
    Connection c = http.getConnection();  
    InstagramAPI api = new InstagramMock(c);  
    return api.getFollowers("john");  
}
```

```
class InstagramMock  
    implements InstagramAPI {  
    void connect() {}  
    List<Node> getFollowers(String name) {  
        if (name.equals("john")) {  
            return List.of(...)  
        }  
    }  
}
```

Let's clone Google Docs

1. What choices should we make for our front end?
 - Which UI architecture?
 - Which technology stack?
 - What libraries comply with our accessibility standards?
2. What should it look like?
 - Who is our user today?
 - What is the most basic user story for Google Docs?
 - What's the easiest mockup/wireframe we can find?
3. Where is the boundary between front end and backend?
 - Where should the document text live?

Mobbing Activity

Join Zoom to see the screen better!

- Mobber will come to the hot seat.
- Notetaker sits in the front of the class with their laptop to take notes in the #mobbing channel.
- Let's write the code that implements the user story for our Google Docs clone.
 - Don't forget to mock the backend!
- Everyone tell the mobber what to say to ChatGPT to get it to create our Google Docs front end.

Code Review

- The LLM just wrote a ton of code.
- But seriously, who wants to read all that?
- Our senior devs ask us:
 - Does this code do what it claims?
 - Are there any programming bugs?
 - Are there any design bugs?
- Can the LLM help us review the code?

Mobbing: LLM role-based code review

- Let's swap the mobber and notetaker.
- Try out these prompts!
 - You are an architect. What is the purpose of this code? Explain the high-level structure and any design patterns used.
 - You are a developer concerned about performance. Show me the performance bottlenecks.
 - You are a tester. Show me where all the test cases are. What parts of the code need more tests?
 - You are a security analyst. Tell me where the code needs to be hardened further against adversaries.

We forgot to look for bugs!

- Prompt: You are a developer concerned about bugs. Create a checklist to check for common mistakes in our front end code. Be sure it includes checks for standards compliance with WCAG and ARIA.
- Prompt: Great checklist. Now verify our code against this checklist and tell me what you found.
- Prompt: For each problem you found, suggest a bug fix. If I think it's good, I'll ask you to apply the fix.

Next class

- Activating your UI
 - Event handlers
 - Modifying the DOM